

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT

On

ANALYSIS AND DESIGN OF ALGORITHMS (23CS4PCADA)

Submitted by

**Jahnavi Verma
1BF24CS126**

**in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING**



**B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)**

BENGALURU-560019

February to June 2026

CERTIFICATE

**B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering**



This is to certify that the Lab work entitled “**ANALYSIS AND DESIGN OF ALGORITHMS**” carried out by Jahnvi Verma (1BF24CS126), who is bonafide student of **B. M. S. College of Engineering**. It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2025-26. The Lab report has been approved as it satisfies the academic requirements in respect of Analysis and Design of Algorithms Lab - **(23CS4PCADA)** work prescribed for the said degree.

Manjula S
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1	a. Write program to obtain the Topological ordering of vertices in a given digraph. b. Leetcode	4
2	Implement Johnson Trotter algorithm to generate permutations.	9
3	a. Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort. b. Leetcode	13
4	a. Sort a given set of N integer elements using Quick Sort technique and compute its time taken. b. Leetcode	19
5	a. Sort a given set of N integer elements using Heap Sort technique and compute its time taken. b. Leetcode	24
6	a. Implement 0/1 Knapsack problem using dynamic programming. (5 Marks) b. Leetcode	29
7	Implement All Pair Shortest paths problem using Floyd's algorithm.	34
8	(a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm. (b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.	38
9	a. Implement Fractional Knapsack using Greedy technique. b. Leetcode	43
10	From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.	46
11	Implement "N-Queens Problem" using Backtracking.	50

Course outcomes:

CO1	Analyze time complexity of algorithms using asymptotic notations.
CO2	Apply various algorithm design techniques for the given problem.
CO3	Evaluate the appropriate algorithm/design technique for solving a given problem.
CO4	Conduct practical experiments by applying appropriate algorithm design technique to solve problems.

Program 1:-

a) Write program to obtain the Topological ordering of vertices in a given digraph.

CODE:-

```
#include <stdio.h>

#include <stdlib.h>

#define MAX_VERTICES 100

void topologicalSort(int num_vertices, int
adj_matrix[MAX_VERTICES][MAX_VERTICES]) {
    int in_degree[MAX_VERTICES] = {0};
    int topo_order[MAX_VERTICES];
    int queue[MAX_VERTICES];
    int front = 0, rear = 0;
    int count = 0;

    for (int i = 0; i < num_vertices; i++) {
        for (int j = 0; j < num_vertices; j++) {
            if (adj_matrix[i][j] == 1) {
                in_degree[j]++;
            }
        }
    }

    for (int i = 0; i < num_vertices; i++) {
        if (in_degree[i] == 0) {
            queue[rear++] = i;
        }
    }

    while (front < rear) {
```

```

int current_vertex = queue[front++];
topo_order[count++] = current_vertex;

for (int neighbor = 0; neighbor < num_vertices; neighbor++) {
    if (adj_matrix[current_vertex][neighbor] == 1) {
        in_degree[neighbor]--;

        if (in_degree[neighbor] == 0) {
            queue[rear++] = neighbor;
        }
    }
}

if (count != num_vertices) {
    printf("-1\n");
} else {
    for (int i = 0; i < count; i++) {
        printf("%d ", topo_order[i]);
    }
    printf("\n");
}
}

int main() {
    int num_vertices;
    int adj_matrix[MAX_VERTICES][MAX_VERTICES];

    if (scanf("%d", &num_vertices) != 1 || num_vertices > MAX_VERTICES) {

```

```

        return 1;
    }

    for (int i = 0; i < num_vertices; i++) {
        for (int j = 0; j < num_vertices; j++) {
            if (scanf("%d", &adj_matrix[i][j]) != 1) {
                return 1;
            }
        }
    }
}

topologicalSort(num_vertices, adj_matrix);

return 0;
}

```

OUTPUT:-

```

6
0 1 1 0 0 0
0 0 0 1 0 0
0 0 0 1 1 0
0 0 0 0 0 1
0 0 0 0 0 1
0 0 0 0 0 0
0 1 2 3 4 5

```

b) Leet Code exercises on topological sorting.

802. Find Eventual Safe States

Solved

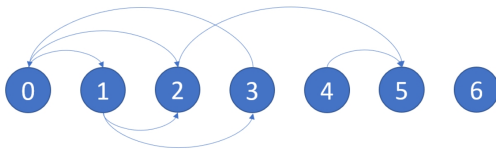
Medium Topics Companies

There is a directed graph of n nodes with each node labeled from 0 to $n - 1$. The graph is represented by a **0-indexed** 2D integer array `graph` where `graph[i]` is an integer array of nodes adjacent to node `i`, meaning there is an edge from node `i` to each node in `graph[i]`.

A node is a **terminal node** if there are no outgoing edges. A node is a **safe node** if every possible path starting from that node leads to a **terminal node** (or another safe node).

Return an array containing all the **safe nodes** of the graph. The answer should be sorted in **ascending order**.

Example 1:



Input: `graph = [[1,2],[2,3],[5],[0],[5],[],[1]]`

Output: `[2,4,5,6]`

Explanation: The given graph is shown above.

Nodes 5 and 6 are terminal nodes as there are no outgoing edges from either of them.

Every path starting at nodes 2, 4, 5, and 6 all lead to either node 5 or 6.

7.1K 285 60 Online

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`graph =
[[1,2],[2,3],[5],[0],[5],[],[1]]`

Output

`[2,4,5,6]`

Expected

`[2,4,5,6]`

Contribute a testcase

```
C v Auto
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4
5
6 typedef struct {
7     int *data;
8     int front, rear;
9 } Queue;
10
11 Queue* createQueue(int n) {
12     Queue* q = (Queue*)malloc(sizeof(Queue));
13     q->data = (int*)malloc(n * sizeof(int));
14     q->front = q->rear = 0;
15     return q;
16 }
17
18 bool isEmpty(Queue* q) {
19     return q->front == q->rear;
20 }
21
22 void enqueue(Queue* q, int val) {
23     q->data[q->rear++] = val;
24 }
25
26 int dequeue(Queue* q) {
27     return q->data[q->front++];
28 }
29
30 int* eventualSafeNodes(int** graph, int graphSize, int* graphColSize, int*
returnSize) {
31     int n = graphSize;
32     int* outdegree = (int*)calloc(n, sizeof(int));
```

Saved

Ln 5, Col 1

All Submissions

Accepted 113 / 113 testcases passed

jahnvi_verma submitted at May 25, 2026 09:50

Analysis

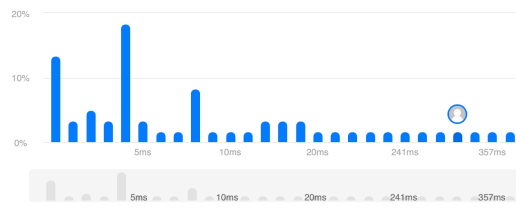
Solution

Runtime

297 ms Beats 13.33%

Memory

33.01 MB Beats 13.33%



Code | C

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <stdbool.h>
4
5 // Queue for BFS
6 typedef struct {
7     int *data;
8     int front, rear;
```

View more

Program 2:-

Implement Johnson Trotter algorithm to generate permutations.

CODE:-

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter n: ");
    scanf("%d", &n);

    int p[n], dir[n];

    for (int i = 0; i < n; i++) {
        p[i] = i + 1;
        dir[i] = -1;
    }

    while (1) {
        for (int i = 0; i < n; i++)
            printf("%d ", p[i]);
        printf("\n");

        int mobile = 0, pos = -1;
        for (int i = 0; i < n; i++) {
            if (dir[i] == -1 && i > 0 && p[i] > p[i - 1]) {
                if (p[i] > mobile) {
                    mobile = p[i];
                    pos = i;
                }
            }
        }
    }
}
```

```

    }

    if (dir[i] == 1 && i < n - 1 && p[i] > p[i + 1]) {
        if (p[i] > mobile) {
            mobile = p[i];
            pos = i;
        }
    }
}

if (mobile == 0)
    break;

int next = pos + dir[pos];

int temp = p[pos];
p[pos] = p[next];
p[next] = temp;

temp = dir[pos];
dir[pos] = dir[next];
dir[next] = temp;

pos = next;
for (int i = 0; i < n; i++) {
    if (p[i] > mobile)
        dir[i] = -dir[i];
}
}

```

```
    return 0;  
}
```

OUTPUT:-

```
Enter n: 4  
1 2 3 4  
1 2 4 3  
1 4 2 3  
4 1 2 3  
4 1 3 2  
1 4 3 2  
1 3 4 2  
1 3 2 4  
3 1 2 4  
3 1 4 2  
3 4 1 2  
4 3 1 2  
4 3 2 1  
3 4 2 1  
3 2 4 1  
3 2 1 4  
2 3 1 4  
2 3 4 1  
2 4 3 1  
4 2 3 1  
4 2 1 3  
2 4 1 3  
2 1 4 3  
2 1 3 4
```

Program 3:-

Sort a given set of N integer elements using Merge Sort technique and compute its time taken. Run the program for different values of N and record the time taken to sort.

CODE:-

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

void merge(int beg, int mid, int end, int a[]) {
    int n1 = mid - beg + 1;
    int n2 = end - mid;
    int a1[n1], a2[n2];

    for (int i = 0; i < n1; i++) {
        a1[i] = a[beg + i];
    }
    for (int j = 0; j < n2; j++) {
        a2[j] = a[mid + 1 + j];
    }

    int i = 0;
    int j = 0;
    int k = beg;

    while (i < n1 && j < n2) {
        if (a1[i] <= a2[j]) {
            a[k] = a1[i];
            i++;
        } else {
```

```
        a[k] = a2[j];  
        j++;  
    }  
    k++;  
}
```

```
while (i < n1) {  
    a[k] = a1[i];  
    i++;  
    k++;  
}
```

```
while (j < n2) {  
    a[k] = a2[j];  
    j++;  
    k++;  
}  
}
```

```
void merge_sort(int beg, int end, int a[]) {  
    if (beg < end) {  
        int mid = (beg + end) / 2;  
        merge_sort(beg, mid, a);  
        merge_sort(mid + 1, end, a);  
        merge(beg, mid, end, a);  
    }  
}
```

```
int main() {
```

```

int a[25000];

srand(time(0));

for (int i = 0; i < 25000; i++) {
    a[i] = rand() % 25000;
}

clock_t start = clock();
merge_sort(0, 24999, a);
clock_t end = clock();

printf("time taken to execute in ms is %f\n", (double)(end - start) * 1000.0 /
CLOCKS_PER_SEC);

for (int i = 0; i < 10; i++) {
    printf("%d\t", a[i]);
}

printf("\n");

return 0;
}

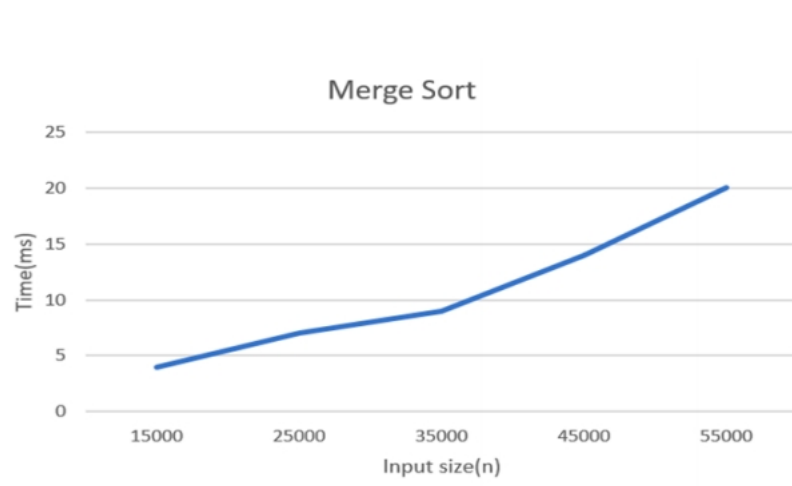
```

OUTPUT:-

```

50000
Size: 50000
Time taken: 0.017451 seconds

```



b) Leet Code exercises on merge sorting.

88. Merge Sorted Array

Solved ✓

Easy Topics Companies Hint

You are given two integer arrays `nums1` and `nums2`, sorted in **non-decreasing order**, and two integers `m` and `n`, representing the number of elements in `nums1` and `nums2` respectively.

Merge `nums1` and `nums2` into a single array sorted in **non-decreasing order**.

The final sorted array should not be returned by the function, but instead be stored *inside the array* `nums1`. To accommodate this, `nums1` has a length of `m + n`, where the first `m` elements denote the elements that should be merged, and the last `n` elements are set to 0 and should be ignored. `nums2` has a length of `n`.

Example 1:

Input: `nums1 = [1,2,3,0,0,0]`, `m = 3`, `nums2 = [2,5,6]`, `n = 3`
Output: `[1,2,2,3,5,6]`
Explanation: The arrays we are merging are `[1,2,3]` and `[2,5,6]`.
The result of the merge is `[1,2,2,3,5,6]` with the underlined elements coming from `nums1`.

Example 2:

Input: `nums1 = [1]`, `m = 1`, `nums2 = []`, `n = 0`
Output: `[1]`
Explanation: The arrays we are merging are `[1]` and `[]`.
The result of the merge is `[1]`.

Example 3:

Input: `nums1 = [0]`, `m = 0`, `nums2 = [1]`, `n = 1`
Output: `[1]`
Explanation: The arrays we are merging are `[]` and `[1]`.

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

`nums1 =`
`[1,2,3,0,0,0]`

`m =`
`3`

`nums2 =`
`[2,5,6]`

`n =`
`3`

Output

`[1,2,2,3,5,6]`

Expected

`[1,2,2,3,5,6]`

Contribute a testcase

```
1 void merge(int* nums1, int nums1Size, int m, int* nums2, int nums2Size, int n) {
2     int j = 0;
3     int i = 0;
4     for (i; i < nums1Size && j < nums2Size; i++) {
5         if (nums1[i] >= nums2[j]) {
6             for (int k = nums1Size - 1; k > i; k--) {
7                 nums1[k] = nums1[k - 1];
8             }
9             nums1[i] = nums2[j];
10            j++;
11        }
12    }
13    if (j < nums2Size) {
14        if (j < nums2Size) {
15            for (int k = m + j; k < m + n; k++, j++) {
16                nums1[k] = nums2[j];
17            }
18        }
19    }
20 }
21
22 }
```

All Submissions

Accepted 59 / 59 testcases passed

jahnvi_verma submitted at Feb 23, 2026 10:42

Analysis

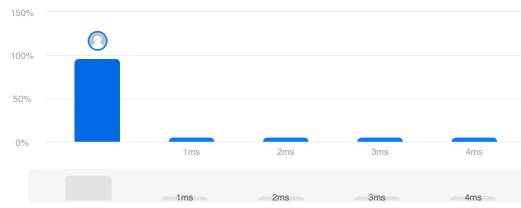
Solution

Runtime

0 ms Beats 100.00%

Memory

10.27 MB Beats 100.00%



Code | C

```
1 void merge(int* nums1, int nums1Size, int m, int* nums2, int nums2Size, int
2     int j = 0;
3     int i = 0;
4     for (i; i < nums1Size && j < nums2Size; i++) {
5         if (nums1[i] >= nums2[j]) {
6             for (int k = nums1Size - 1; k > i; k--) {
7                 nums1[k] = nums1[k - 1];
8             }
9             nums1[i] = nums2[j];
10            j++;
11        }
12    }
13    if (j < nums2Size) {
14        if (j < nums2Size) {
15            for (int k = m + j; k < m + n; k++, j++) {
16                nums1[k] = nums2[j];
17            }
18        }
19    }
20 }
```

View more

Program 4:-

Sort a given set of N integer elements using Quick Sort technique and compute its time taken.

CODE:-

```
#include <stdio.h>

#include <stdlib.h>

#include <time.h>

void quicksort(int low, int high, int arr[]);
int partition(int low, int high, int arr[]);

void quicksort(int low, int high, int arr[]) {
    if (low < high) {
        int p = partition(low, high, arr);
        quicksort(low, p - 1, arr);
        quicksort(p + 1, high, arr);
    }
}

int partition(int low, int high, int arr[]) {
    int i = low;
    int j = high + 1;
    int pivot = arr[low];
    int temp;

    while (i < j) {
        do {
            i++;
        } while (i <= high && arr[i] <= pivot);
```

```

do {
    j--;
} while (arr[j] > pivot);

if (i < j) {
    temp = arr[i];
    arr[i] = arr[j];
    arr[j] = temp;
}
}

temp = arr[j];
arr[j] = pivot;
arr[low] = temp;
return j;
}

void main() {
    clock_t start, end;
    double cpu_time_used;
    int n;

    printf("Enter number of elements:");
    scanf("%d", &n);

    int arr[n];
    printf("Enter elements:");
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
}

```

```
}
```

```
start = clock();
```

```
quicksort(0, n - 1, arr);
```

```
end = clock();
```

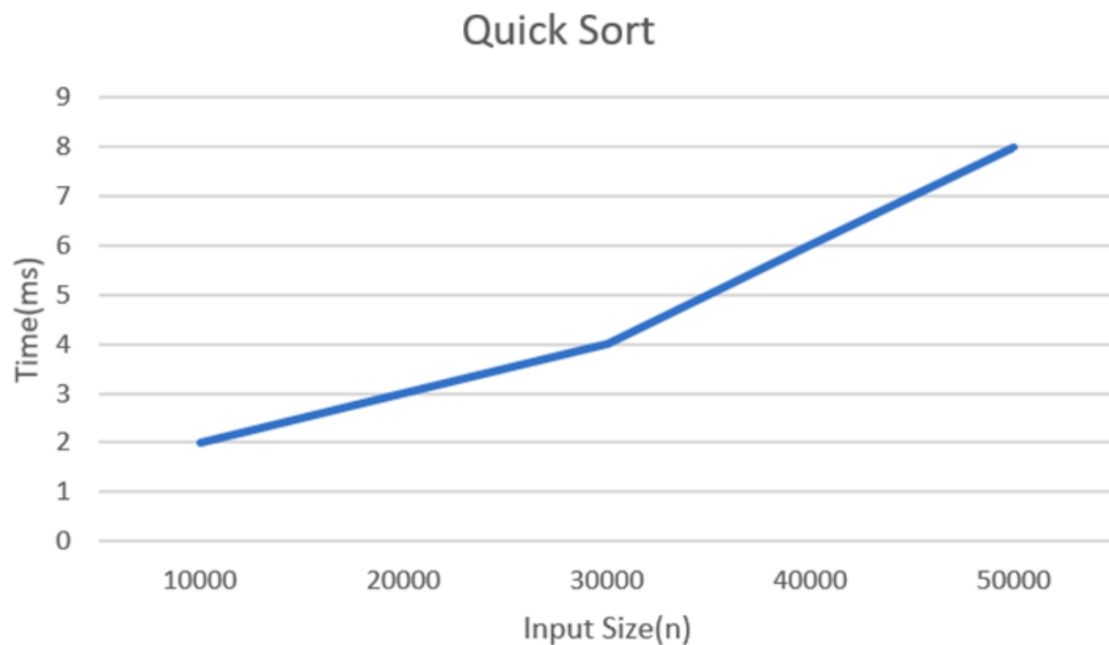
```
cpu_time_used = (((double)(end - start)) / CLOCKS_PER_SEC) * 1000.0;
```

```
printf("Sorted array:");
```

```
}
```

OUTPUT:-

```
Enter number of elements:5
Enter elements:5 4 3 2 1
Sorted array:1 2 3 4 5
Execution time:0.01100 ms
```



b) Leet Code exercises on quick sorting.

1636. Sort Array by Increasing Frequency

Solved

Easy Topics Companies Hint

Given an array of integers `nums`, sort the array in **increasing** order based on the frequency of the values. If multiple values have the same frequency, sort them in **decreasing** order.

Return the *sorted array*.

Example 1:

Input: `nums = [1,1,2,2,2,3]`

Output: `[3,1,1,2,2,2]`

Explanation: '3' has a frequency of 1, '1' has a frequency of 2, and '2' has a frequency of 3.

Example 2:

Input: `nums = [2,3,1,3,2]`

Output: `[1,3,3,2,2]`

Explanation: '2' and '3' both have a frequency of 2, so they are sorted in decreasing order.

Example 3:

Input: `nums = [-1,1,-6,4,5,-6,1,4,1]`

Output: `[5,-1,4,4,-6,-6,1,1,1]`

Constraints:

- `1 <= nums.length <= 100`

Accepted Runtime: 0 ms

Case 1 Case 2 Case 3

Input

`nums =
[1,1,2,2,2,3]`

Output

`[3,1,1,2,2,2]`

Expected

`[3,1,1,2,2,2]`

Contribute a testcase

```
1 /**
2  * Note: The returned array must be malloced, assume caller calls free().
3  */
4  int compare(const void* a, const void* b, void* count) {
5      int valA = *(int*)a;
6      int valB = *(int*)b;
7      int* cnt = (int*)count;
8
9      if (cnt[valA + 100] == cnt[valB + 100]) {
10         return valB - valA;
11     }
12     return cnt[valA + 100] - cnt[valB + 100];
13 }
14 int* frequencySort(int* nums, int numsSize, int* returnSize) {
15     int hashmap[200] = {0};
16     for (int i = 0; i < numsSize; i++) {
17         hashmap[(nums[i] + 100)]++;
18     }
19     qsort_r(nums, numsSize, sizeof(int), compare, count);
20
21     *returnSize = numsSize;
22     return nums;
23 }
24 }
```

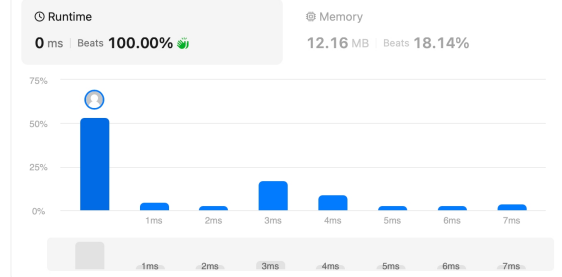
All Submissions

Accepted 180 / 180 testcases passed

jahnvi_verma submitted at Mar 02, 2026 10:04

Analysis

Solution



Code | C

```
1 /**
2  * Note: The returned array must be malloced, assume caller calls free().
3  */
4  struct node{
5      int value;
6      int frequency;
7  }node[100];
8  int* frequencySort(int* nums, int numsSize, int* returnSize) {
9      // ...
10 }
```

View more

Program 5:-

Sort a given set of N integer elements using Heap Sort technique and compute its time taken.

CODE:-

```
#include <stdio.h>

#include <time.h>

void heapify(int arr[], int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;

    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        int temp = arr[i];
        arr[i] = arr[largest];
        arr[largest] = temp;

        heapify(arr, n, largest);
    }
}

void heapSort(int arr[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
}
```

```

    for (int i = n - 1; i > 0; i--) {
        int temp = arr[0];
        arr[0] = arr[i];
        arr[i] = temp;

        heapify(arr, i, 0);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int n;

    printf("Enter number of elements: ");
    scanf("%d", &n);

    int arr[n];

    printf("Enter elements:\n");
    for (int i = 0; i < n; i++)
        scanf("%d", &arr[i]);

    clock_t start, end;

```

```
start = clock();
heapSort(arr, n);
end = clock();

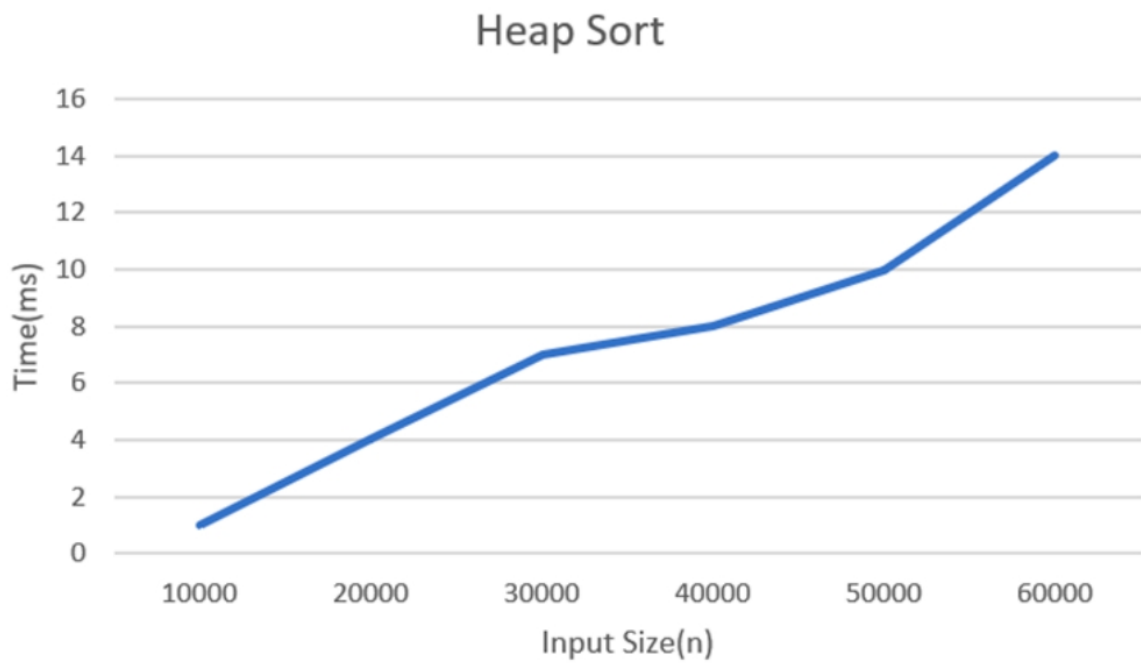
printf("Sorted array:\n");
printArray(arr, n);

double time_taken = ((double)(end - start)) / CLOCKS_PER_SEC;
printf("Time taken: %f seconds\n", time_taken);

return 0;
}
```

OUTPUT:-

```
Enter number of elements: 5
Enter elements:
6 7 4 9 3
Sorted array:
3 4 6 7 9
Time taken: 0.000007 seconds
```



Program 6:-

a) Implement 0/1 Knapsack problem using dynamic programming.

CODE:-

```
#include <stdio.h>

int max(int a, int b) {
    if (a > b)
        return a;
    else
        return b;
}

int knapsack(int W, int wt[], int val[], int n) {
    if (n == 0 || W == 0)
        return 0;

    if (wt[n - 1] > W)
        return knapsack(W, wt, val, n - 1);
    else
        return max(val[n - 1] + knapsack(W - wt[n - 1], wt, val, n - 1), knapsack(W, wt, val, n - 1));
}

int main() {
    int val[] = {3, 4, 5, 6};
    int wt[] = {2, 3, 4, 5};
    int W = 8;
    int n = sizeof(val) / sizeof(val[0]);

    printf("%d\n", knapsack(W, wt, val, n));

    return 0;
}
```

}

OUTPUT:-10

b)Leet Code exercise

821. Shortest Distance to a Character

Solved

Easy Topics Companies

Given a string `s` and a character `c` that occurs in `s`, return an array of integers `answer` where `answer.length == s.length` and `answer[i]` is the distance from index `i` to the closest occurrence of character `c` in `s`.

The distance between two indices `i` and `j` is `abs(i - j)`, where `abs` is the absolute value function.

Example 1:

Input: `s = "loveleetcode", c = "e"`

Output: `[3,2,1,0,1,0,0,1,2,2,1,0]`

Explanation: The character 'e' appears at indices 3, 5, 6, and 11 (0-indexed).
The closest occurrence of 'e' for index 0 is at index 3, so the distance is `abs(0 - 3) = 3`.
The closest occurrence of 'e' for index 1 is at index 3, so the distance is `abs(1 - 3) = 2`.
For index 4, there is a tie between the 'e' at index 3 and the 'e' at index 5, but the distance is still the same: `abs(4 - 3) == abs(4 - 5) = 1`.
The closest occurrence of 'e' for index 8 is at index 6, so the distance is `abs(8 - 6) = 2`.

Example 2:

Input: `s = "aaab", c = "b"`

Output: `[3,2,1,0]`

Constraints:

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`s =`

`"loveleetcode"`

`c =`

`"e"`

Output

`[3,2,1,0,1,0,0,1,2,2,1,0]`

Expected

`[3,2,1,0,1,0,0,1,2,2,1,0]`

Contribute a testcase

```
1 /**
2  * Note: The returned array must be malloced, assume caller calls free().
3  */
4
5 int* shortestToChar(char* s, char c, int* returnSize) {
6     int n = strlen(s);
7     *returnSize = n;
8     int* result = (int*)malloc(n * sizeof(int));
9
10    int prev = -1;
11
12    for (int i = 0; i < n; i++) {
13        if (s[i] == c) prev = i;
14        if (prev == -1) {
15            result[i] = INT_MAX;
16        } else {
17            result[i] = i - prev;
18        }
19    }
20
21    prev = -1;
22
23    for (int i = n - 1; i >= 0; i--) {
24        if (s[i] == c) prev = i;
25        if (prev != -1) {
26            int dist = prev - i;
27            if (dist < result[i]) {
28                result[i] = dist;
29            }
30        }
31    }
32
33    return result;
34 }
```

All Submissions

Accepted 76 / 76 testcases passed

jahnavi_verma submitted at May 04, 2026 09:59

Analysis

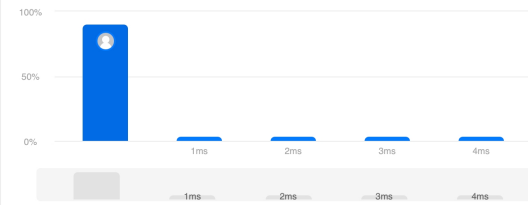
Solution

Runtime

0 ms Beats 100.00%

Memory

11.64 MB Beats 53.12%



Code | C

```
1 /**
2  * Note: The returned array must be malloced, assume caller calls free().
3  */
4
5 int* shortestToChar(char* s, char c, int* returnSize) {
6     int n = strlen(s);
7     *returnSize = n;
8     int* result = (int*)malloc(n * sizeof(int));
9
10    int prev = -1;
11
12    for (int i = 0; i < n; i++) {
13        if (s[i] == c) prev = i;
14        if (prev == -1) {
15            result[i] = INT_MAX;
16        } else {
17            result[i] = i - prev;
18        }
19    }
20
21    prev = -1;
22
23    for (int i = n - 1; i >= 0; i--) {
24        if (s[i] == c) prev = i;
25        if (prev != -1) {
26            int dist = prev - i;
27            if (dist < result[i]) {
28                result[i] = dist;
29            }
30        }
31    }
32
33    return result;
34 }
```

Program 7:-

Implement All Pair Shortest paths problem using Floyd's algorithm.

CODE:-

```
#include <stdio.h>

#define V 4
#define INF 99999

void floydWarshall(int graph[V][V]) {
    int dist[V][V];

    for (int i = 0; i < V; i++) {
        for (int j = 0; j < V; j++) {
            dist[i][j] = graph[i][j];
        }
    }

    for (int k = 0; k < V; k++) {
        for (int i = 0; i < V; i++) {
            for (int j = 0; j < V; j++) {
                if (dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }

    printf("Shortest distance matrix:\n");

    for (int i = 0; i < V; i++) {
```

```

        for (int j = 0; j < V; j++) {
            if (dist[i][j] == INF)
                printf("INF ");
            else
                printf("%d ", dist[i][j]);
        }
        printf("\n");
    }
}

```

```

int main() {
    int graph[V][V] = {
        {0, 5, INF, 10},
        {INF, 0, 3, INF},
        {INF, INF, 0, 1},
        {INF, INF, INF, 0}
    };

    floydWarshall(graph);

    return 0;
}

```

OUTPUT:-

```

Shortest distance matrix:
0 5 8 9
INF 0 3 4
INF INF 0 1
INF INF INF 0

```

Program 8:-

a) Find Minimum Cost Spanning Tree of a given undirected graph using Prim's algorithm.

CODE:-

```
#include <stdio.h>
#include <limits.h>

#define V 5

int minKey(int key[], int mstSet[]) {
    int min = INT_MAX, min_index;

    for (int v = 0; v < V; v++) {
        if (mstSet[v] == 0 && key[v] < min) {
            min = key[v];
            min_index = v;
        }
    }

    return min_index;
}

void printMST(int parent[], int graph[V][V]) {
    printf("Edge Weight\n");
    for (int i = 1; i < V; i++) {
        printf("%d - %d %d\n", parent[i], i, graph[i][parent[i]]);
    }
}

void primMST(int graph[V][V]) {
    int parent[V];
    int key[V];
```

```

int mstSet[V];

for (int i = 0; i < V; i++) {
    key[i] = INT_MAX;
    mstSet[i] = 0;
}

key[0] = 0;
parent[0] = -1;

for (int count = 0; count < V - 1; count++) {
    int u = minKey(key, mstSet);
    mstSet[u] = 1;

    for (int v = 0; v < V; v++) {
        if (graph[u][v] && mstSet[v] == 0 && graph[u][v] < key[v]) {
            parent[v] = u;
            key[v] = graph[u][v];
        }
    }
}

printMST(parent, graph);
}

int main() {
    int graph[V][V] = {
        {0, 2, 0, 6, 0},
        {2, 0, 3, 8, 5},
        {0, 3, 0, 0, 7},
        {6, 8, 0, 0, 9},
        {0, 5, 7, 9, 0}
    }
}

```

```
};  
  
    primMST(graph);  
    return 0;  
}
```

OUTPUT:-

```
Edge Weight  
0 - 1 2  
1 - 2 3  
0 - 3 6  
1 - 4 5
```

b) Find Minimum Cost Spanning Tree of a given undirected graph using Kruskal's algorithm.

CODE:-

```
#include <stdio.h>

#include <stdlib.h>

struct Edge {
    int u, v, w;
};

int find(int parent[], int i) {
    if (parent[i] == i) return i;
    return find(parent, parent[i]);
}

int cmp(const void* a, const void* b) {
    return ((struct Edge*)a)->w - ((struct Edge*)b)->w;
}

int main() {
    int V = 4;
    int E = 5;

    struct Edge edges[] = {
        {0, 1, 10},
        {0, 2, 6},
        {0, 3, 5},
        {1, 3, 15},
        {2, 3, 4}
    };
};
```

```

qsort(edges, E, sizeof(edges[0]), cmp);

int parent[V];
for (int i = 0; i < V; i++) parent[i] = i;

printf("Edges in MST:\n");
int totalWeight = 0, count = 0;

for (int i = 0; i < E && count < V - 1; i++) {
    int u = edges[i].u;
    int v = edges[i].v;
    int w = edges[i].w;

    int pu = find(parent, u);
    int pv = find(parent, v);

    if (pu != pv) {
        printf("%d -- %d == %d\n", u, v, w);
        totalWeight += w;
        parent[pu] = pv;
        count++;
    }
}

printf("Total weight of MST: %d\n", totalWeight);
return 0;
}

```

OUTPUT:-

```
Edges in MST:  
2 -- 3 == 4  
0 -- 3 == 5  
0 -- 1 == 10  
Total weight of MST: 19
```

Program 9:-

a) Implement Fractional Knapsack using Greedy technique.

CODE:-

```
#include <stdio.h>

#include <stdlib.h>

#define max 100

float knapsack(float capacity, float weight[max], float profit[max], float ratio[max], int n) {
    int i = 0;
    float total_profit = 0;

    while (i < n && capacity > 0) {
        if (weight[i] <= capacity) {
            capacity -= weight[i];
            total_profit += profit[i];
            i++;
        } else {
            total_profit += ratio[i] * capacity;
            break;
        }
    }

    return total_profit;
}

float weight[max], profit[max], ratio[max];

int main() {
```

```

int n;

float capacity;

printf("enter number of objects:");
scanf("%d", &n);

for (int i = 0; i < n; i++) {
    printf("enter profit of obj %d:", i);
    scanf("%f", &profit[i]);
    printf("enter weight of obj %d:", i);
    scanf("%f", &weight[i]);
    ratio[i] = profit[i] / weight[i];
}

int swapped;
float temp;

do {
    swapped = 0;
    for (int i = 0; i < n - 1; i++) {
        if (ratio[i] < ratio[i + 1]) {
            temp = ratio[i];
            ratio[i] = ratio[i + 1];
            ratio[i + 1] = temp;

            temp = profit[i];
            profit[i] = profit[i + 1];
            profit[i + 1] = temp;
        }
    }
} while (swapped);

```

```

        temp = weight[i];
        weight[i] = weight[i + 1];
        weight[i + 1] = temp;

        swapped = 1;
    }
}
} while (swapped);

printf("enter capacity:");
scanf("%f", &capacity);

float total_profit = knapsack(capacity, weight, profit, ratio, n);
printf("Total profit = %.2f\n", total_profit);

return 0;
}

```

OUTPUT:-

```

enter number of objects:3
enter profit of obj 0:60
enter weight of obj 0:10
enter profit of obj 1:100
enter weight of obj 1:20
enter profit of obj 2:120
enter weight of obj 2:30
enter capacity:50
Total profit = 240.00

```

b) Leet Code exercise

1710. Maximum Units on a Truck

Solved

Easy Topics Companies Hint

You are assigned to put some amount of boxes onto **one** truck. You are given a 2D array `boxTypes`, where `boxTypes[i] = [numberOfBoxesi, numberOfUnitsPerBoxi]`:

- `numberOfBoxesi` is the number of boxes of type `i`.
- `numberOfUnitsPerBoxi` is the number of units in each box of the type `i`.

You are also given an integer `truckSize`, which is the **maximum** number of **boxes** that can be put on the truck. You can choose any boxes to put on the truck as long as the number of boxes does not exceed `truckSize`.

Return the **maximum** total number of **units** that can be put on the truck.

Example 1:

Input: `boxTypes = [[1,3],[2,2],[3,1]]`, `truckSize = 4`
Output: 8

Explanation: There are:

- 1 box of the first type that contains 3 units.
- 2 boxes of the second type that contain 2 units each.
- 3 boxes of the third type that contain 1 unit each.

You can take all the boxes of the first and second types, and one box of the third type.

The total number of units will be $= (1 * 3) + (2 * 2) + (1 * 1) = 8$.

Example 2:

Input: `boxTypes = [[5,10],[2,5],[4,7],[3,9]]`, `truckSize = 10`
Output: 91

Accepted Runtime: 0 ms

Case 1 Case 2

Input

`boxTypes =`
`[[1,3],[2,2],[3,1]]`

`truckSize =`

4

Output

8

Expected

8

Contribute a testcase

C Auto

```
1 int compare(const void *a, const void *b) {
2     int *boxA = *(int **)a;
3     int *boxB = *(int **)b;
4     return boxB[1] - boxA[1]; // descending order
5 }
6
7 int maximumUnits(int** boxTypes, int boxTypesSize, int* boxTypesColSize, int
truckSize) {
8     // Sort boxTypes by units per box
9     qsort(boxTypes, boxTypesSize, sizeof(int*), compare);
10
11     int totalUnits = 0;
12     for (int i = 0; i < boxTypesSize && truckSize > 0; i++) {
13         int boxesToTake = (boxTypes[i][0] < truckSize) ? boxTypes[i][0] :
truckSize;
14         totalUnits += boxesToTake * boxTypes[i][1];
15         truckSize -= boxesToTake;
16     }
17     return totalUnits;
18 }
19
```

← All Submissions

Accepted 77 / 77 testcases passed

jahnavi_verma submitted at Apr 13, 2026 10:28

Analysis

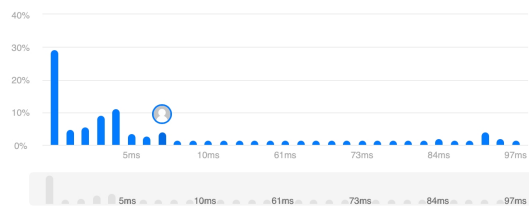
Solution

Runtime

7 ms Beats 33.57%

Memory

11.97 MB Beats 35.66%



Code | C

```
1 int compare(const void *a, const void *b) {
2     int *boxA = *(int **)a;
3     int *boxB = *(int **)b;
4     return boxB[1] - boxA[1]; // descending order
5 }
6
7 int maximumUnits(int** boxTypes, int boxTypesSize, int* boxTypesColSize, int
truckSize) {
8     // Sort boxTypes by units per box
9     qsort(boxTypes, boxTypesSize, sizeof(int*), compare);
10
11     int totalUnits = 0;
12     for (int i = 0; i < boxTypesSize && truckSize > 0; i++) {
13         int boxesToTake = (boxTypes[i][0] < truckSize) ? boxTypes[i][0] :
truckSize;
14         totalUnits += boxesToTake * boxTypes[i][1];
15         truckSize -= boxesToTake;
16     }
17     return totalUnits;
18 }
19
```

View more

Program 10:-

From a given vertex in a weighted connected graph, find shortest paths to other vertices using Dijkstra's algorithm.

CODE:-

```
#include <stdio.h>

#include <stdlib.h>

#include <limits.h>

#define INF 999

#define n 5

int min(int d[], int visited[]) {
    int min = INT_MAX;
    int min_index = -1;

    for (int i = 0; i < n; i++) {
        if (visited[i] == 0 && d[i] <= min) {
            min = d[i];
            min_index = i;
        }
    }

    return min_index;
}

void dijkstra(int cost[n][n], int source) {
    int dist[n];
    int visited[n];
```

```

for (int i = 0; i < n; i++) {
    dist[i] = INT_MAX;
    visited[i] = 0;
}

dist[source] = 0;

for (int j = 0; j < n - 1; j++) {
    int u = min(dist, visited);
    visited[u] = 1;

    for (int v = 0; v < n; v++) {
        if (visited[v] != 1 && cost[u][v] != INF && dist[u] != INT_MAX &&
            dist[u] + cost[u][v] < dist[v]) {
            dist[v] = dist[u] + cost[u][v];
        }
    }
}

printf("\nShortest paths from vertex %d:\n", source);
for (int i = 0; i < n; i++) {
    printf("%d -> %d = %d\n", source, i, dist[i]);
}

}

int main() {
    int i, j, source;
    int cost[n][n];

```

```

printf("Enter cost adjacency matrix (%dx%d):\n", n, n);
for (i = 0; i < n; i++) {
    for (j = 0; j < n; j++) {
        scanf("%d", &cost[i][j]);
        if (cost[i][j] == 0 && i != j)
            cost[i][j] = INF;
    }
}

printf("Enter source vertex (0-%d): ", n - 1);
scanf("%d", &source);

dijkstra(cost, source);

return 0;
}

```

OUTPUT:-

```

Enter cost adjacency matrix (4x4):
0 1 4 0
1 0 2 5
4 2 0 1
0 5 1 0
Enter source vertex (0-3): 0

Shortest paths from vertex 0:
0 -> 0 = 0
0 -> 1 = 1
0 -> 2 = 3
0 -> 3 = 4

```

Program 11:-

Implement “N-Queens Problem” using Backtracking.

CODE:-

```
#include <stdio.h>

int board[20];

int n;

int isSafe(int row, int col) {
    for (int i = 1; i < row; i++) {
        if (board[i] == col ||
            (i - board[i] == row - col) ||
            (i + board[i] == row + col))
            return 0;
    }
    return 1;
}

void solve(int row) {
    if (row > n) {
        printf("\nSolution:\n");
        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= n; j++) {
                if (board[i] == j)
                    printf("Q ");
                else
                    printf(". ");
            }
            printf("\n");
        }
    }
}
```

```

    }
    return;
}

for (int col = 1; col <= n; col++) {
    if (isSafe(row, col)) {
        board[row] = col;
        solve(row + 1);
    }
}
}

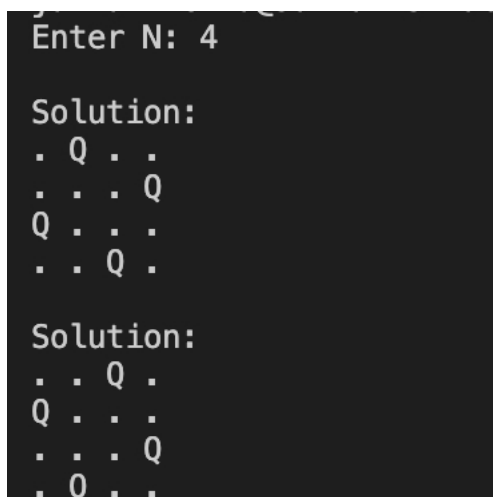
int main() {
    printf("Enter N: ");
    scanf("%d", &n);

    solve(1);

    return 0;
}

```

OUTPUT:-



```

Enter N: 4

Solution:
. Q . .
. . . Q
Q . . .
. . Q .

Solution:
. . Q .
Q . . .
. . . Q
. Q . .

```

